ENTERPRISE DIGITAL BANKING SYSTEM

08

Project and Maintenance Documentation

Project overview, requirements, business rules, developer setup guide, coding conventions, known limitations, roadmap and changelog - everything a new developer needs to continue the project.

Document	08 — Project and Maintenance Documentation
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. Project Overview
2. 2. Requirements
3. 3. Business Rules
4. 4. Developer Setup Guide
5. 5. Known Limitations
6. 6. Roadmap
7. 7. Changelog
8. 8. Maintaining This Documentation

1. Project Overview

Enterprise Digital Banking System is a Spring Boot backend that simulates a retail bank: JWT-secured users, customers with KYC, accounts, deposits/withdrawals/transfers on a double-entry ledger, beneficiaries, loans with EMI schedules and a complete audit trail. A separate **notification-service** consumes Kafka events and notifies customers.

Item	Value
Repositories	`Enterprise-Digital-Banking-System` (banking-app, port 8080) · `notification-service` (port 8081)
Language / framework	Java 17, Spring Boot 4.1, Maven wrapper
Data	MySQL 8 (`banking_db` , `notification_db`)
Messaging	Apache Kafka 3.8 (KRaft)
Size today	10 feature modules, 9 tables, 40 REST endpoints, 2 Kafka topics, 38 tests
Main branch	`master`

1.1 Documentation set

#	Document	Read it when you need to...
01	Architecture	understand components, layers, deployment and design decisions (ADRs)
02	Database	see the ER diagram, columns, constraints, indexes, migrations
03	Flows and interactions	follow what happens on login, transfer, loan approval, EMI payment
04	API	call an endpoint: request, response, errors
05	Security	check auth, roles, sensitive data, threats, audit policy
06	Testing	run or write tests
07	Deployment and operations	run, deploy, monitor, back up, troubleshoot
08	Project and maintenance (this)	get started and continue the work

2. Requirements

2.1 Functional requirements

ID	Requirement	Status
FR-01	A visitor can register with name, username, e-mail, phone and password.	BUILT
FR-02	A user can log in with e-mail and password and receive a JWT.	BUILT
FR-03	A user can refresh tokens, change, reset and recover a password, and log out.	PLANNED
FR-04	Staff can create, view, update, list and delete customer profiles linked to users.	BUILT
FR-05	Staff can complete KYC with a unique PAN and Aadhaar.	BUILT
FR-06	Staff can open SAVINGS / CURRENT accounts with a generated unique account number.	BUILT
FR-07	Staff can update, freeze and close accounts (close only at zero balance).	BUILT
FR-08	Deposit to and withdraw from an ACTIVE account.	BUILT
FR-09	Transfer between ACTIVE accounts by account id or by an ACTIVE beneficiary.	BUILT
FR-10	Every money movement produces ledger DEBIT/CREDIT lines with running balance.	BUILT
FR-11	View transaction by id / reference, account history and ledger.	BUILT
FR-12	Customers can add, rename, activate and remove beneficiaries.	BUILT
FR-13	Customers can apply for loans; the EMI is calculated on application.	BUILT
FR-14	Managers can approve (disburse + EMI schedule) or reject loans.	BUILT
FR-15	Customers can pay EMIs; paying the last EMI closes the loan.	BUILT
FR-16	Paying an EMI debits the customer's account via a transaction and ledger entries.	PLANNED
FR-17	Overdue EMIs are flagged automatically each day.	PLANNED
FR-18	Every significant action is recorded in an audit log searchable by entity or user.	BUILT (loan/EMI/customer events planned)

FR-19	Customers are notified (e-mail) of transfers and beneficiary activation.	**BUILT** (SMTP creds deferred)
FR-20	Cards: issue, block, set limits for an account.	**PLANNED**
FR-21	List endpoints support pagination, sorting and customer name search.	**PLANNED**

2.2 Non-functional requirements

ID	Category	Requirement
NFR-01	Consistency	A money movement is atomic: balances, transaction, ledger and audit commit together or not at all.
NFR-02	Accuracy	Money uses BigDecimal / DECIMAL(18,2); no floating-point values are stored.
NFR-03	Concurrency	Concurrent updates to one account never lose an update (optimistic locking).
NFR-04	Security	Passwords hashed with BCrypt; stateless JWT; no secrets in source; role-based access (planned).
NFR-05	Availability	A notification outage never fails a banking operation.
NFR-06	Performance	p95 < 250 ms for transfers at 50 req/s; account reads cached (planned).
NFR-07	Auditability	Who, what, when, from which IP, for every state change; retained ≥ 8 years.
NFR-08	Maintainability	Package-by-feature, consistent layering, ≥ 80 % service-layer test coverage.
NFR-09	Portability	Runs with `docker compose up`; configuration by environment variables.
NFR-10	Observability	Health, metrics and dashboards via Actuator, Prometheus, Grafana (planned).

3. Business Rules

ID	Area	Rule	Enforced in
BR-01	Users	E-mail is unique; password and confirmation must match; new users are CUSTOMER and ACTIVE.	UserServiceImpl
BR-02	Customers	One customer profile per user; phone, PAN and Aadhaar are unique across customers.	CustomerServiceImpl + unique constraints
BR-03	KYC	Completing KYC sets the customer VERIFIED and the user's `kycVerified` flag; deleting the customer resets it.	CustomerServiceImpl
BR-04	Accounts	Account numbers are 12 digits starting with "10", randomly generated, unique.	AccountServiceImpl
BR-05	Accounts	Opening balance ≥ 0. New accounts are ACTIVE.	DTO validation, entity default
BR-06	Accounts	A CLOSED account cannot be updated, frozen or closed again. Freezing a FROZEN account is rejected.	AccountServiceImpl
BR-07	Accounts	An account can be closed only when its balance is exactly zero.	AccountServiceImpl
BR-08	Transactions	Minimum amount is 1.00; remarks ≤ 255 chars.	DTO validation
BR-09	Transactions	Only ACTIVE accounts can send or receive money.	TransactionServiceImpl
BR-10	Transactions	Withdrawals and transfers cannot exceed the available balance (no overdraft).	TransactionServiceImpl
BR-11	Transfers	Exactly one of destination account / beneficiary; source ≠ destination.	TransactionServiceImpl
BR-12	Transfers	A beneficiary used for a transfer must be ACTIVE and belong to the source account's customer.	TransactionServiceImpl
BR-13	Ledger	Deposit = 1 CREDIT; withdrawal = 1 DEBIT; transfer = 1 DEBIT + 1 CREDIT of equal amount. Each line stores the balance after posting.	LedgerServiceImpl
BR-14	Transactions	Every transaction has a unique reference "TXN" + 13 digits.	TransactionServiceImpl
BR-15	Beneficiaries	The same account cannot be added twice by one customer. New beneficiaries are PENDING and must be activated before use.	BeneficiaryServiceImpl
BR-16	Loans	Principal ≥ 1 000; interest rate ≥ 0.01 % p.a.; tenure 1–360 months.	DTO validation
BR-17	Loans	EMI = $P \cdot r \cdot (1+r)^n / ((1+r)^n - 1)$, r = annual rate / 12 / 100, rounded HALF_UP to 2 decimals.	LoanServiceImpl
BR-18	Loans	Only PENDING loans can be approved or rejected. Approval credits the principal to the disbursement account and creates one EMI per month, due monthly from the approval date.	LoanServiceImpl, EmiServiceImpl

BR-19	EMIs	An EMI can be paid once. When no unpaid EMI remains, the loan becomes CLOSED.	EmiServiceImpl
BR-20	Notifications	Transfers and beneficiary activations emit events; each event is notified at most once (idempotency key = eventId).	Producers, notification-service
BR-21	Audit	Audit entries never contain passwords and are never modified or deleted.	AuditServiceImpl

4. Developer Setup Guide

4.1 Tools

Tool	Version	Notes
JDK	17+	Temurin or Corretto
IDE	IntelliJ IDEA	Enable annotation processing (Lombok plugin)
MySQL	8.0	Local install or `docker run -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=... mysql:8.0`
Docker Desktop	latest	Kafka, Redis, later the full stack
Postman / curl	—	API calls
Git	—	

4.2 First run

```
git clone <repo-url> Enterprise-Digital-Banking-System
git clone <repo-url> notification-service          # sibling folder

# environment (IntelliJ: Run > Edit Configurations > Environment variables)
DB_PASSWORD=<mysql password>
JWT_SECRET=<output of: openssl rand -base64 48>

cd notification-service && docker compose up -d      # Kafka on localhost:9092
cd ../Enterprise-Digital-Banking-System
./mvnw spring-boot:run                               # http://localhost:8080
./mvnw test                                          # see doc 06 - fix compile errors first
```

Then follow the quick-start script in document 04 §11 to register, log in and make a transfer.

4.3 Adding a new feature module (checklist)

1. Create package `<feature>/` with `entity`, `repository`, `dto/request`, `dto/response`, `mapper`, `service` + `service/impl`, `controller`.
2. Entity: `@Entity @Table(name = "plural_snake")`, Lombok builder, `@ManyToOne(fetch = LAZY)` to parents, `@Enumerated(STRING)`, money `precision = 18, scale = 2`, `@PrePersist` for `createdAt` and default status.
3. Request DTOs with Bean Validation and clear messages; response DTOs never expose entities or sensitive fields.
4. Service interface + `@Service @RequiredArgsConstructor @Transactional` impl; `readOnly = true` on queries; throw `ResourceNotFoundException` / `BadRequestException`.
5. Call `auditService.log(...)` on every state change.
6. Never inject another module's service if it already depends on yours — use its repository (ADR-005).
7. Controller under `/api/v1/<plural>`; 201 for create, 204 for delete.
8. Add a Flyway migration (once Flyway is in), unit tests for every branch, and update documents 02 and 04.

4.4 Coding conventions

Topic	Convention

Injection	Constructor injection via `@RequiredArgsConstructor` and `final` fields
Naming	`XxxController`, `XxxService` / `XxxServiceImpl`, `XxxRepository`, `XxxMapper`, `CreateXxxRequest`, `XxxResponse`
Exceptions	Domain exceptions only; mapping to HTTP lives in `GlobalExceptionHandler`
Tests	`method_condition_expectedResult`, e.g. `withdraw_insufficientBalance_throwsBadRequestException`
Commits	Imperative, describe the change (“Add EMI payment endpoint”); one logical change per commit
Branching	Feature branches → pull request → `master` with green CI

5. Known Limitations

#	Limitation	Impact	Planned fix
L-01	**Test sources do not compile** since Kafka producers were added to service constructors	`./mvnw test` fails	Add producer mocks to 3 test classes (doc 06 §1)
L-02	No role-based authorization or ownership checks	Any logged-in user can approve loans, read others' accounts and audit logs (IDOR)	`@PreAuthorize` + ownership service (doc 05 §3)
L-03	Paying an EMI does not debit any account	Loans can be closed without repayment	Debit via TransactionService + ledger + audit
L-04	No overdue EMI detection	OVERDUE never set	`@Scheduled` daily job
L-05	Loan / EMI / customer actions are not audited	Incomplete trail	Add `auditService.log` calls
L-06	Kafka event published before DB commit, fire-and-forget	Possible notification for a rolled-back transfer; lost events if broker is down	Transactional outbox / `AFTER_COMMIT` listener
L-07	Deposits and withdrawals emit no event	No notification for them	Publish `transaction.completed` for all types
L-08	Account opening does not require verified KYC	Compliance gap	Check `kycStatus == VERIFIED`
L-09	No token revocation / refresh; 24 h tokens; user status not checked at login	Blocked users keep access	Refresh tokens, status check, deny-list
L-10	No rate limiting	Brute-force on login	Nginx + RateLimitFilter (roadmap 09)
L-11	Optimistic-lock conflicts and malformed JSON return 500	Misleading status	Handlers mapping to 409 / 400
L-12	List endpoints are unpaged	Slow at scale	Pageable (roadmap 06)
L-13	Schema managed by `ddl-auto=update`	Unreviewed schema changes	Flyway (roadmap 04)
L-14	Customer delete is a hard delete and fails with children	500 error	Soft delete / 409
L-15	Last EMI is not adjusted for rounding	Total repaid may differ from exact amortisation by a few paise	Compute final installment as remaining balance
L-16	Card module is an empty skeleton; `CreateEmiRequest`, `AuthResponse`, `TokenResponse` are empty classes	Dead code	Implement or delete
L-17	`lastLogin`, `emailVerified`, `phoneVerified` are never updated	Unused fields	Update on login / verification flows

L-18	SMTP credentials deliberately not configured	E-mail notifications end in FAILED	Set `MAIL_USERNAME` / `MAIL_PASSWORD` (Gmail app password)

6. Roadmap

Step	Reference doc	Deliverable
0	—	Fix test compilation; close loan/EMI gaps (L-03, L-04, L-05)
1	roadmap-refs/01-redis	Redis cache on account reads with eviction on every write path
2	roadmap-refs/02-kafka-live-and-advanced	Run Kafka end-to-end; dead-letter topics; consumer groups; lag monitoring
3	roadmap-refs/03-logging-monitoring	Logback config, Actuator, Prometheus, Grafana
4	roadmap-refs/04-docker-deployment-infra	Dockerfile, full docker-compose, dev/prod profiles, Flyway, indexes
5	roadmap-refs/05-testing	Service, repository, controller and integration tests
6	roadmap-refs/06-pagination-sorting-search-swagger	Pageable history, customer search, Swagger UI
7	roadmap-refs/07-security-performance	Security checklist, optimistic-lock test, N+1 review
8	roadmap-refs/09-reverse-proxy-rate-limiting	Nginx edge + rate limiting (optional)
9	roadmap-refs/08-deployment-and-wrapup	Production build, AWS Elastic Beanstalk + RDS, README, final review
Later	—	Role-based authorization, refresh tokens, cards module, transactional outbox

7. Changelog

Derived from the git history of `master` (64 commits). Dates are commit dates.

Date	Version	Changes
2026-10-13	0.9	EMI module completed: entity, repository, schedule generation, payment, loan auto-close, EmiController. Documentation set (this folder) added.
2026-10-09 – 10-12	0.8	Loan module: entity, types/status, EMI formula, apply/approve/reject endpoints; error fixes; README update.
2026-10-05 – 10-06	0.7	Embedded notification module removed in favour of notification-service. Kafka producer config, `TransactionCompletedEvent`, `BeneficiaryActivatedEvent`, producers wired into transfer and beneficiary activation.
2026-09-09 – 09-13	0.6	Beneficiary module (CRUD, activation, audit) with unit tests; transfers via beneficiary; Notification/Card/Loan packages scaffolded.
2026-08-31 – 09-01	0.5	Double-entry ledger; transaction and ledger unit tests; AuditLog entity, service, controller.
2026-08-20 – 08-26	0.4	Account APIs (create, update, freeze, close); Transaction entity, deposit/withdraw/transfer service and controller; refactoring.
2026-07-30 – 08-20	0.3	Customer module: DTOs, repository, service, controller, KYC.
2026-07-23 – 07-28	0.2.1	Custom exceptions and GlobalExceptionHandler.
2026-07-05 – 07-22	0.2	User entity, register/login DTOs and logic, BCrypt encoder, SecurityFilterChain, JwtUtil, JwtAuthenticationFilter, entry point.
2026-06-24 – 06-28	0.1	Project created; README and architecture diagram; first Account entity.

> Going forward, keep a `CHANGELOG.md` in the repository root using the *Keep a Changelog* format (Added / Changed / Fixed / Removed per version) and tag releases (`git tag v1.0.0`).

8. Maintaining This Documentation

- Sources: `docs/whole-documentation/source/content/*.html` (one file per PDF), shared look in `source/template.html`, diagrams written in Mermaid inside the HTML.
- Rebuild all PDFs: `python docs/whole-documentation/source/build.py`; rebuild one: `python docs/whole-documentation/source/build.py 04`. Requires Chrome or Edge; Mermaid is bundled locally (works offline).
- When a **PLANNED** item is implemented, change its badge to **BUILT** and update the affected sections.
- Update docs 02 (schema) and 04 (API) in the same commit as the code change.